

Dual Port RAM Verification

Special Assignment Report

Course: 3EC605ME24 : Testing and Verification of Digital Circuits

Submitted by:

23BEC176	23BEC177
Vensi Savani	Aditi Shah



Department of Electronics and Communication Engineering,
Institute of Technology,
Nirma University,
Ahmedabad 382 481

[March – 2026]

1 Abstract

This report presents the design and verification of a Dual-Port Random Access Memory (DPRAM) that supports simultaneous read and write operations through two independent ports, namely Port A and Port B. The proposed design enables concurrent memory access, thereby improving system performance and data throughput in applications requiring parallel processing. Each port is equipped with its own address, data input, data output, and write enable control signals, allowing independent and simultaneous operations. A key feature of the design is its conflict resolution mechanism when both ports access the same memory address simultaneously. In such cases, a deterministic priority scheme is implemented, where the data from Port A is written into the memory, ensuring predictable and consistent behavior. The functionality of the memory is verified using a comprehensive testbench that includes various test scenarios such as independent read/write operations, simultaneous access, and same-address conflicts. Simulation results confirm that the memory operates correctly under all conditions, with proper handling of concurrent operations and priority-based conflict resolution. This design demonstrates the effectiveness of True Dual-Port RAM in high-performance digital systems.

2 Keywords

Dual-Port RAM, Concurrent Memory Access, Read-Write Conflict, Memory Arbitration, FPGA Design, Digital System Verification, Parallel Processing, Hardware Design.

3 Introduction

Memory plays a crucial role in modern digital systems, acting as an interface between different functional units that operate at varying speeds. In many applications such as processors, communication systems, and embedded systems, there is a need for efficient and simultaneous access to memory. Dual-Port Random Access Memory (DPRAM) addresses this requirement by allowing two independent ports to access the same memory array concurrently.

A Dual-Port RAM consists of two fully independent ports, typically referred to as Port A and Port B. Each port has its own address lines, data input and output lines, and control signals such as write enable. This architecture enables both ports to perform read and write operations simultaneously without interfering with each other, provided they access different memory locations.

However, when both ports attempt to access the same memory address at the same time, a conflict arises. To resolve this, a priority-based mechanism is implemented in the proposed design. In this work, Port A is given higher priority over Port B. Therefore, if both ports attempt to write to the same address simultaneously, only the data from Port A is stored in memory, ensuring deterministic and predictable behavior.

The objective of this project is to design such a True Dual-Port RAM and verify its functionality under various operating conditions. The verification process includes testing independent operations of each port, simultaneous read and write operations, and

conflict scenarios involving the same address. The results demonstrate that the design successfully supports concurrent access while maintaining data integrity through a well-defined priority scheme.

This study highlights the importance of dual-port memory architectures in improving system efficiency and provides a practical approach to handling access conflicts in digital memory systems.

4 Literature Review

Efficient memory design is a fundamental requirement in modern digital systems, particularly in applications involving parallel data processing. Dual-port memory architectures have been widely adopted to enhance memory bandwidth and enable simultaneous access to shared data.

Cummings [1] introduced key design techniques for FIFO and memory structures, emphasizing reliable handling of simultaneous read and write operations. The work highlights synchronization and efficient memory utilization, which are essential in dual-port memory systems.

Sutherland et al. [2] provided a comprehensive understanding of SystemVerilog for hardware design and verification. Their work explains concurrent execution in hardware, which forms the basis of independent port operations in Dual-Port RAM.

Bergeron [3] focused on verification methodologies using SystemVerilog, including structured testbench development and assertion-based verification. These techniques are crucial for validating concurrent memory operations and detecting conflicts.

According to Harris and Harris [4], memory systems in digital design must efficiently support parallel access while maintaining data integrity. Their work discusses memory organization and highlights the importance of handling simultaneous operations in modern processors.

Weste and Harris [5] described VLSI design principles, including memory architecture and performance optimization. They emphasized the need for deterministic behavior in shared memory systems, especially when multiple access requests occur simultaneously.

In this project, a Dual-Port RAM is implemented with a priority-based conflict resolution mechanism, where Port A is given precedence over Port B when both attempt to write to the same address. This approach ensures deterministic operation and aligns with established design practices in digital memory systems.

5 Proposed Solution and Methodology

5.1 Design Overview

The proposed design implements a True Dual-Port Random Access Memory (TDPRAM) that allows simultaneous access through two independent ports, namely Port A and Port B. Each port can perform read or write operations independently on the same shared memory array.

A key feature of the design is the implementation of a priority-based conflict resolution mechanism. When both ports attempt to write to the same memory address simultaneously, Port A is given higher priority. Hence, only the data from Port A is written into the memory, ensuring deterministic and predictable behavior.

5.2 Design Parameters

The design is parameterized to allow flexibility and scalability. The key parameters are as follows:

- **Memory Depth:** 8 locations (addresses 0 to 7)
- **Data Width:** 8 bits per memory location
- **Address Width:** 3 bits (since $2^3 = 8$)
- **Clock:** Single synchronous clock for both ports
- **Write Enables:** Separate write enable signals for Port A and Port B

The memory is implemented as a register array:

`mem[0:7]`

5.3 Error Detection Mechanism

To ensure correctness during simulation, two error counters are used:

- **error1:** Tracks mismatches related to Port A operations
- **error2:** Tracks mismatches related to Port B operations

During verification, the expected output is compared with the actual output. If a mismatch occurs, the corresponding error counter is incremented. At the end of simulation, the total number of errors is evaluated:

- If $(error1 + error2) = 0$, the design is considered correct
- Otherwise, the design has functional errors

5.4 RTL Design Strategy

The RTL design is implemented using synchronous logic triggered on the rising edge of the clock. The following strategies are used:

- **Memory Array:** A register-based memory array stores data values.
- **Port A Operation:**
 - If `we_a = 1`, data is written to `mem[add_a]`
 - Data is always available at `read_a`
- **Port B Operation:**
 - If `we_b = 1`, data is written to `mem[add_b]`
 - Data is available at `read_b`
- **Simultaneous Operations:**
 - Both ports can operate simultaneously without interference if different addresses are accessed
- **Conflict Resolution:**
 - If `add_a = add_b` and both write enables are active, only Port A writes to memory
 - Port B write is ignored in this condition

5.5 Verification Strategy

A comprehensive testbench is developed to verify the functionality of the design under different scenarios. The verification strategy includes:

- **Reset Verification:** Ensures memory initializes correctly
- **Independent Operations:**
 - Write and read using Port A
 - Write and read using Port B
- **Simultaneous Operations:**
 - Simultaneous read from both ports
 - Simultaneous write to different addresses
 - One port reads while the other writes
- **Conflict Testing:**
 - Both ports writing to the same address
 - Verification of Port A priority

- **Boundary Testing:**

- Lowest and highest memory addresses

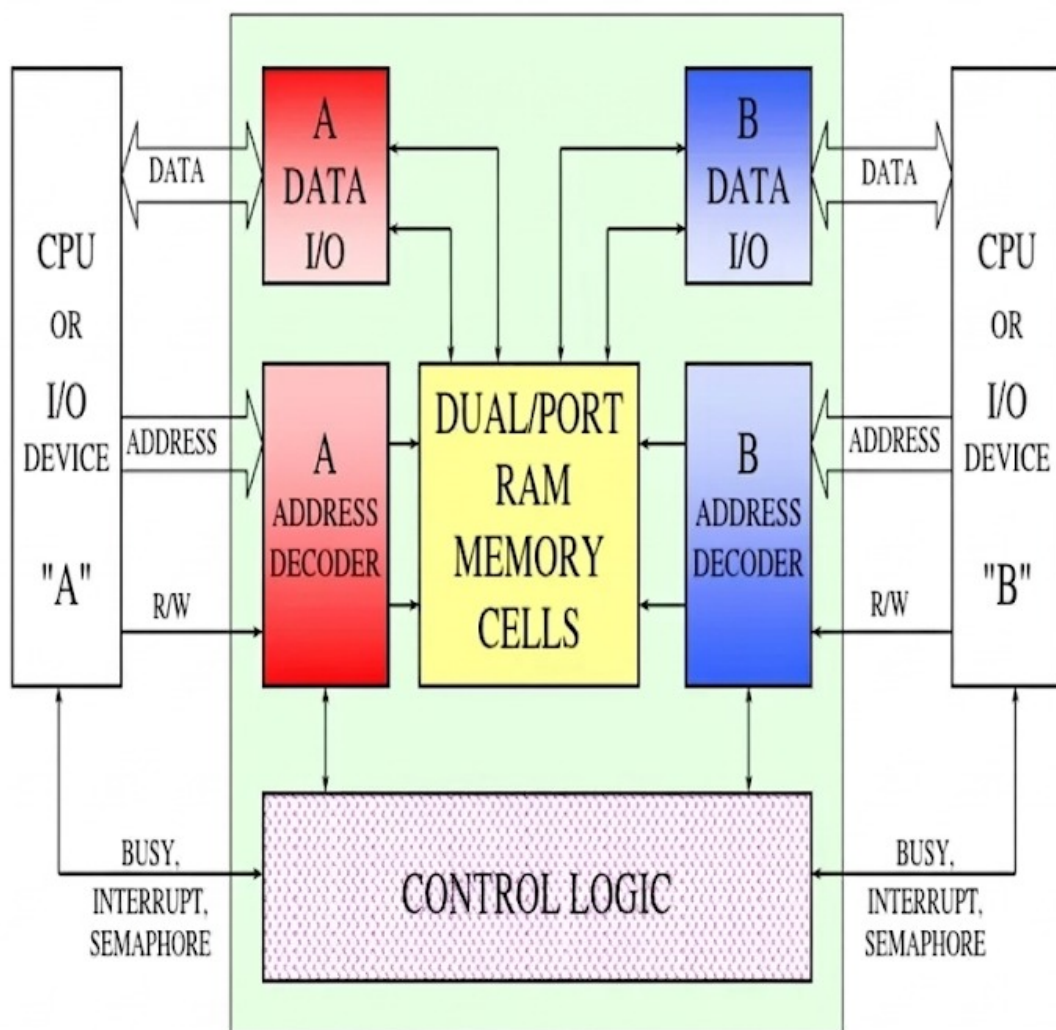
- **Result Checking:**

- Outputs are compared with expected values
- Error counters (`error1`, `error2`) track mismatches

The simulation is considered successful if all test cases pass without any mismatches, confirming correct operation of the True Dual-Port RAM under all conditions.

6 Block Diagram and Flowchart

DUAL-PORT MEMORY BLOCK DIAGRAM



1

Figure 1: Block Diagram of Dual-Port RAM

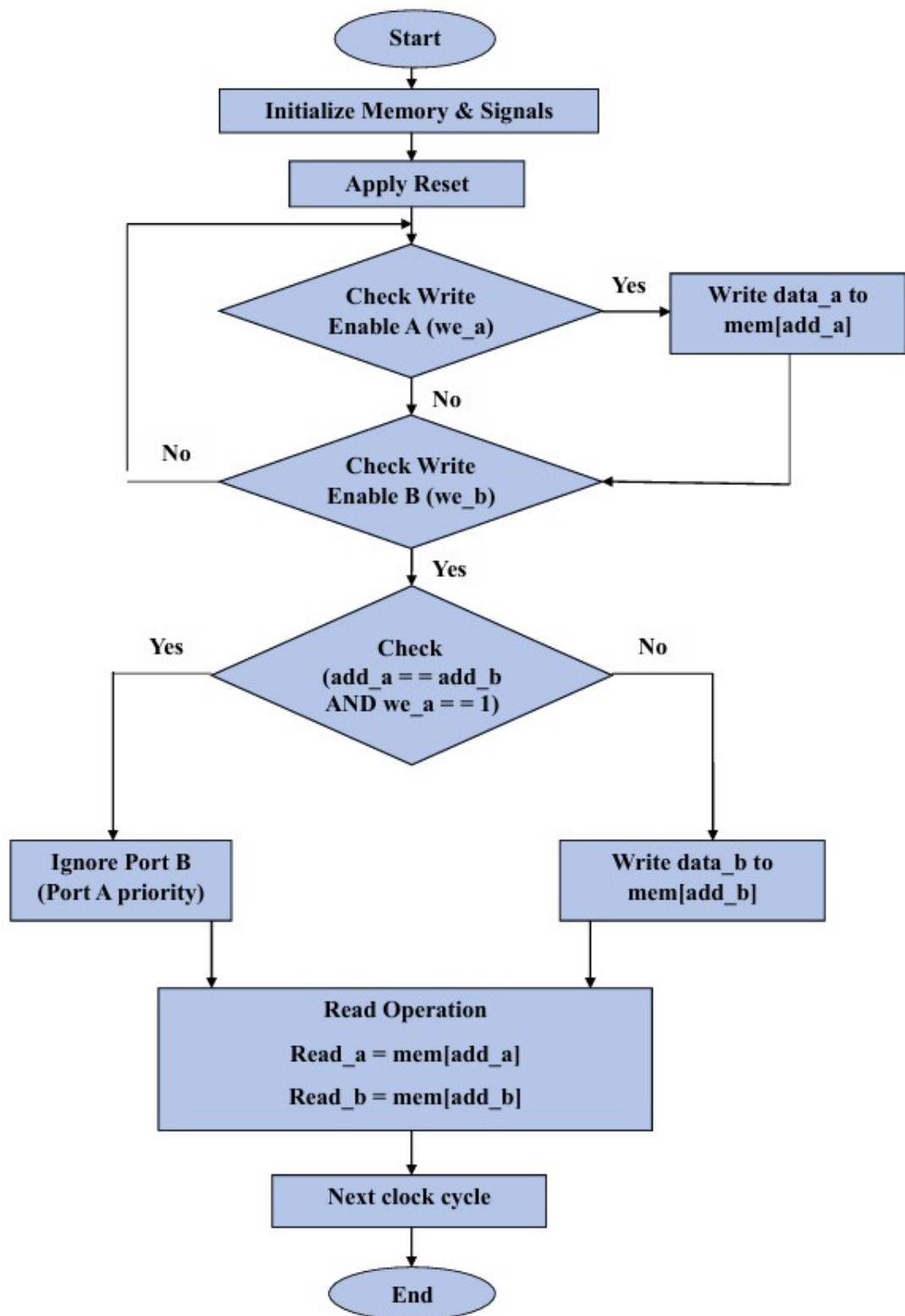


Figure 2: Flowchart of Dual-Port RAM Operation

The diagram illustrates the operation of a Dual-Port RAM. Initially, the memory and control signals are initialized, followed by a reset operation.

The system first checks the write enable signal of Port A. If enabled, data from Port A

is written to the specified memory address. Then, the write enable of Port B is checked. If Port B is active, a condition is evaluated to detect whether both ports are accessing the same address while Port A is also writing.

If such a conflict occurs, Port B is ignored, giving priority to Port A. Otherwise, data from Port B is written to memory. After the write operations, both ports perform read operations independently. The process repeats for every clock cycle.

7 Simulation Results

7.1 Reset Operation

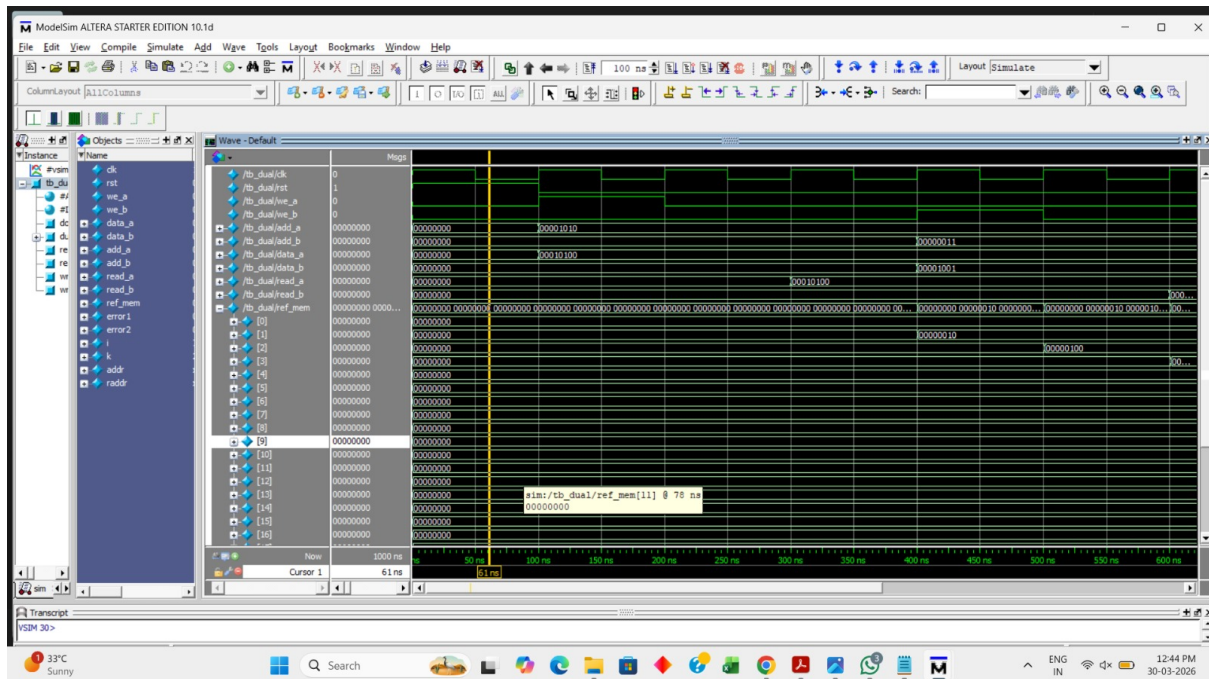


Figure 3: Reset

7.2 Write A and Read A

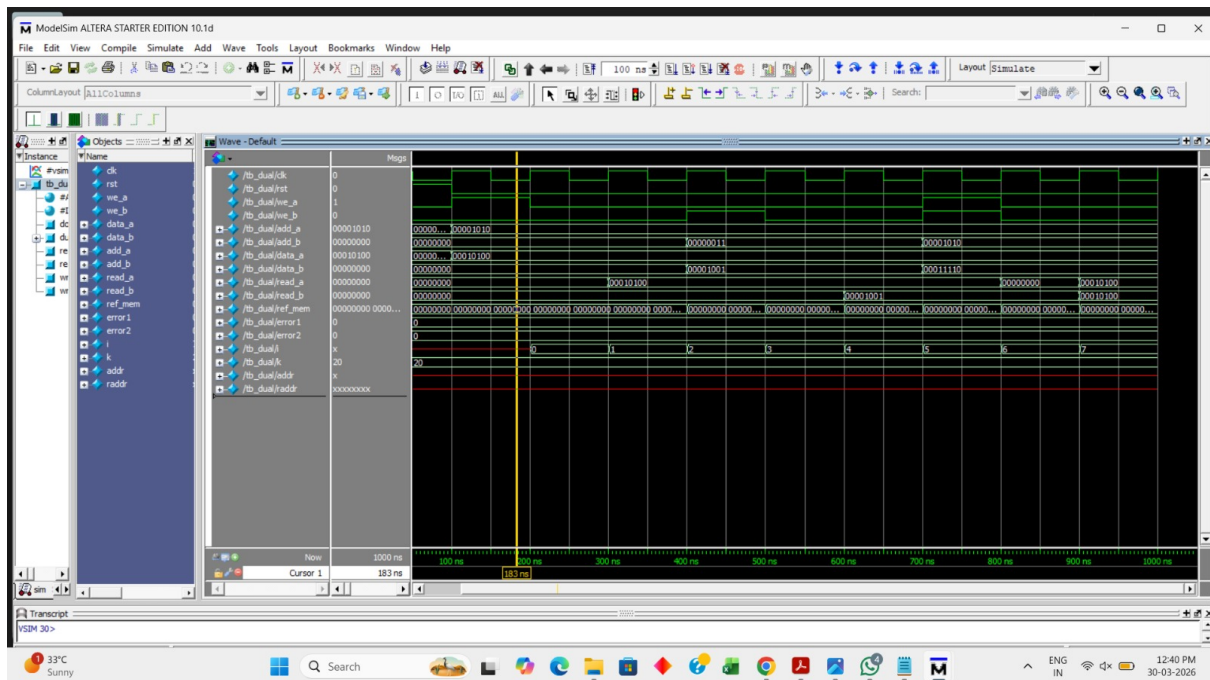


Figure 4: Write A

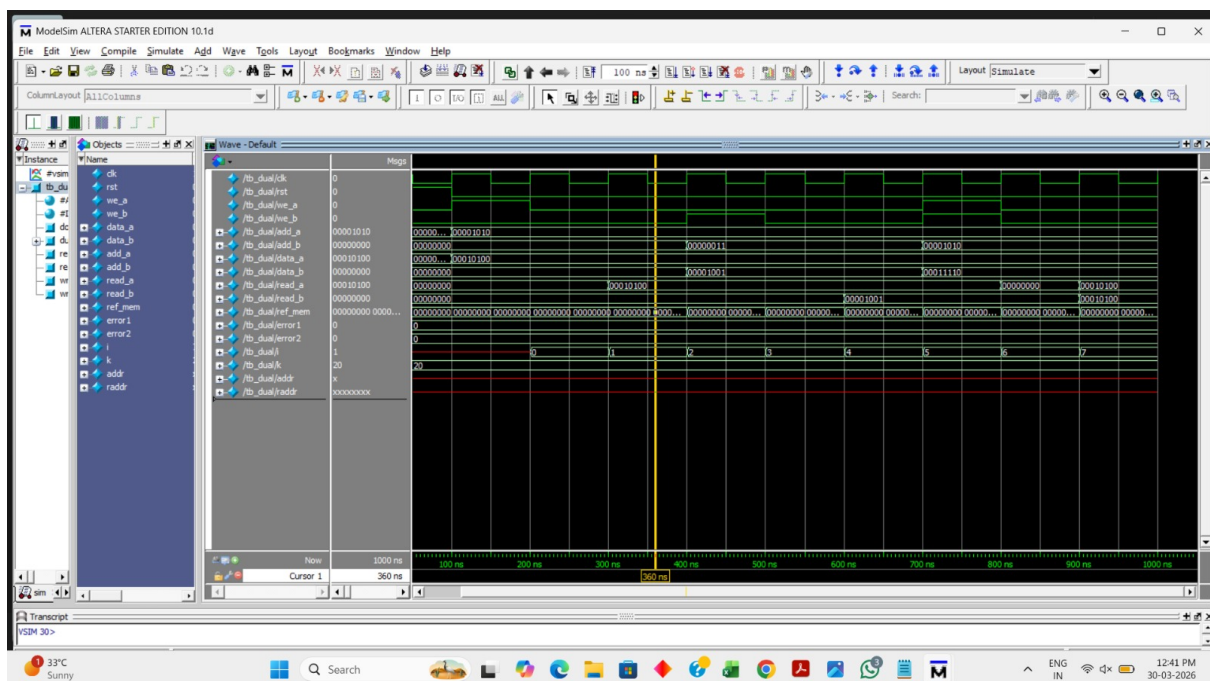


Figure 5: Read A

7.3 Write B and Read B

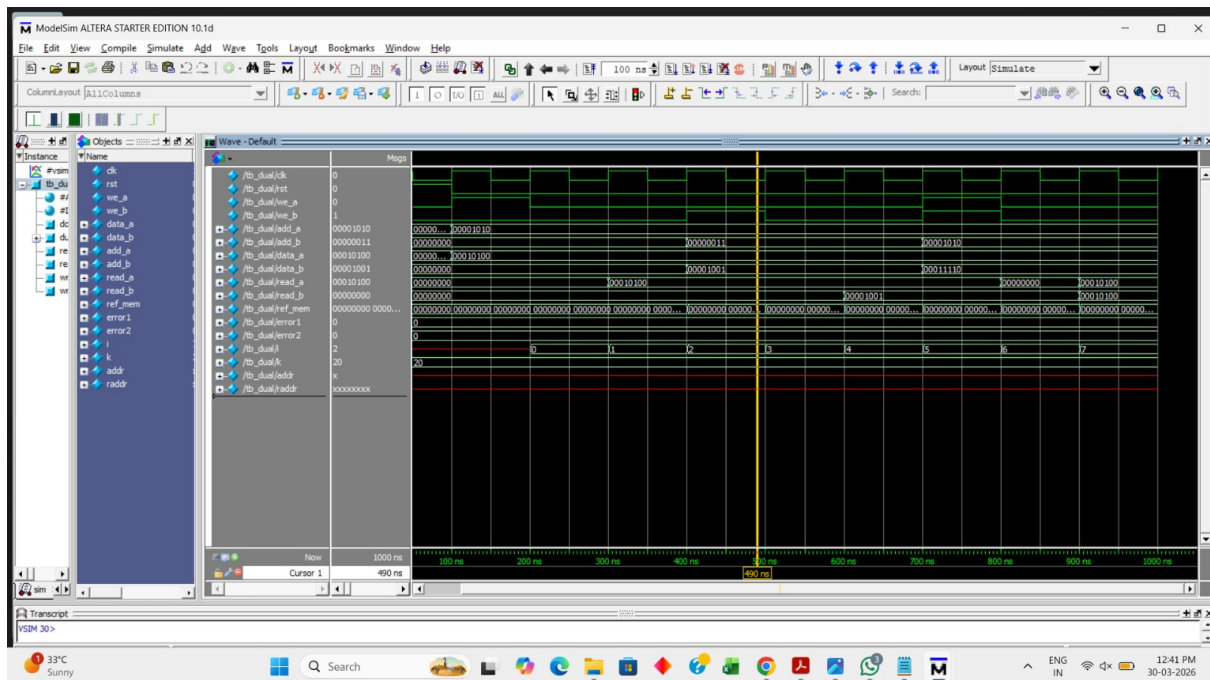


Figure 6: Write B

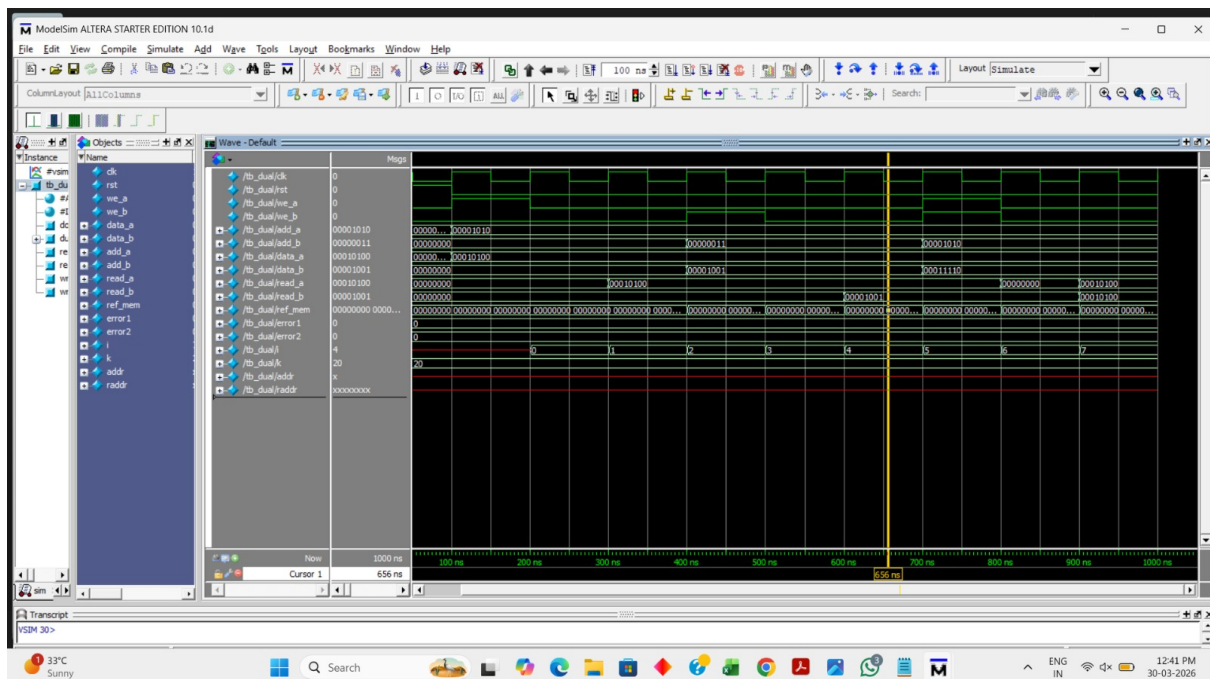


Figure 7: Read B

7.4 Write A and B and Read A and B with same address

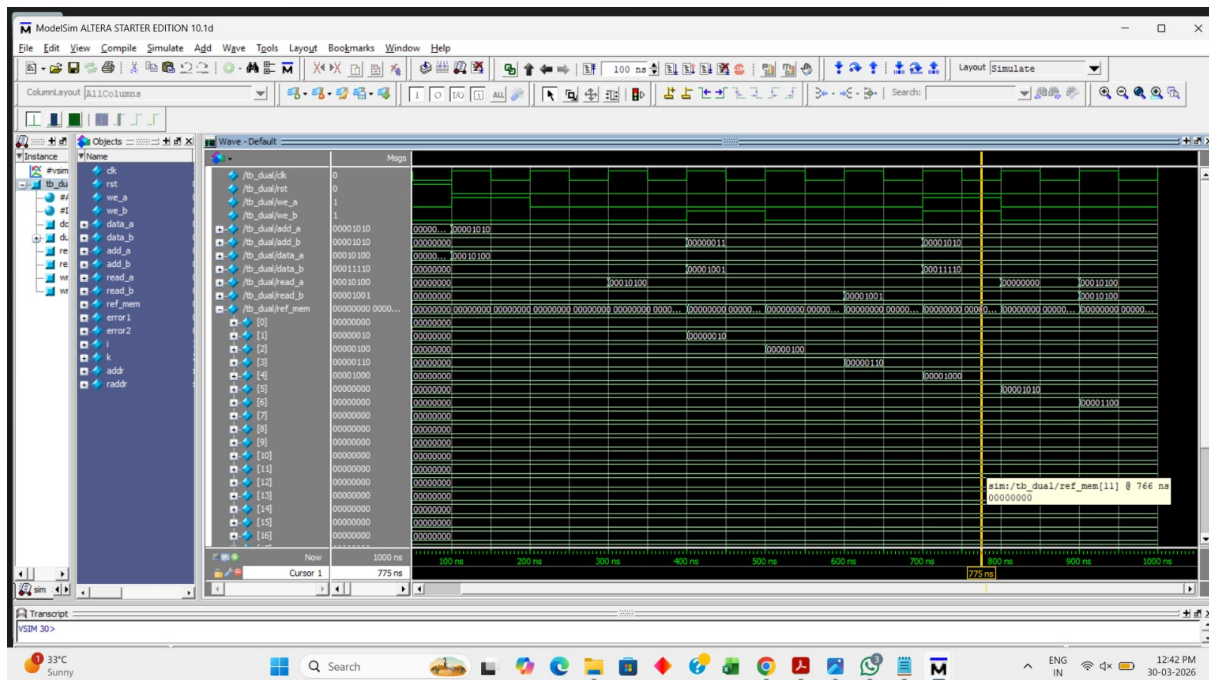


Figure 8: Write A and B both with same address

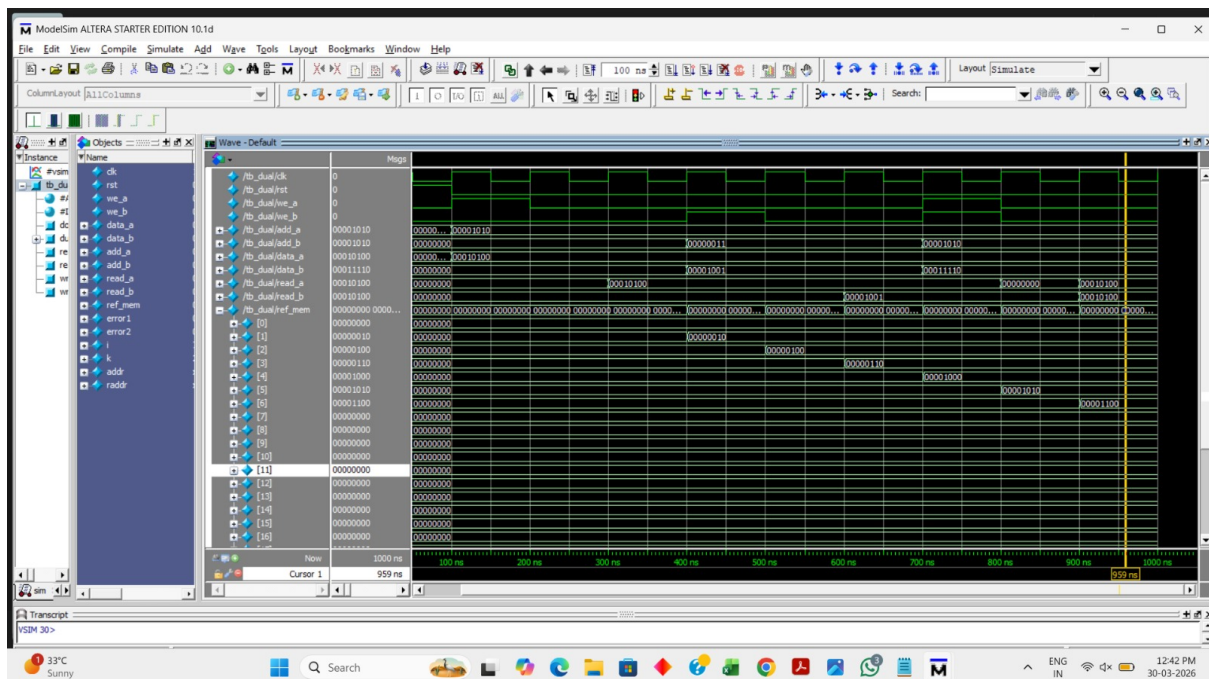


Figure 9: Read A and B both with same address(Port A data wins)

8 Conclusion

In this work, a Dual-Port Random Access Memory (DPRAM) was successfully designed and verified. The proposed design enables simultaneous read and write operations through two independent ports, thereby improving memory access efficiency and supporting parallel processing.

A priority-based conflict resolution mechanism was implemented to handle simultaneous access to the same memory address. In such cases, Port A was given higher priority, ensuring deterministic and reliable behavior of the memory system.

The design was verified using a comprehensive testbench that covered various scenarios including independent operations, simultaneous read/write operations, and same-address conflicts. The use of error counters (error1 and error2) helped in detecting mismatches and validating correctness of the design.

Simulation results confirmed that the memory operates correctly under all test conditions, with proper handling of concurrent access and conflict resolution. Hence, the proposed True Dual-Port RAM design is efficient, reliable, and suitable for high-performance digital systems.

References

- [1] C. E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design,” Sunburst Design Inc., SNUG San Jose, 2002. Available: <http://www.sunburst-design.com/papers>
- [2] S. Sutherland, S. Davidmann, and P. Flake, *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*, Springer, 2006.
- [3] J. Bergeron, *Writing Testbenches Using SystemVerilog*, Springer, 2006.
- [4] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2012.
- [5] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson, 2011.

9 Appendix

Listing 1: RTL Design Code

```
1 module dual(clk, rst, we_a, we_b, data_a, data_b,
2           add_a, add_b, read_a, read_b);
3
4     input          clk, rst, we_a, we_b;
5     input [7:0]    data_a, data_b;
6     input [7:0]    add_a,  add_b;
7     output reg [7:0] read_a, read_b;
8
9     integer i, j;
10    reg [7:0] mem [0:19];
11
12    // Port A
13    always @(posedge clk) begin
14        if (rst) begin
15            read_a <= 8'h00;
16            for (i = 0; i < 20; i = i + 1)
17                mem[i] <= 8'h00;
18        end
19        else begin
20            if (we_a) begin
21                mem[add_a] <= data_a;
22                read_a <= 8'h00;
23            end else begin
24                read_a <= mem[add_a];
25            end
26        end
27    end
28
29    // Port B
30    always @(posedge clk) begin
31        if (rst)
32            begin
33                read_b <= 8'h00;
34                for (j = 0; j < 20; j = j + 1)
35                    mem[j] <= 8'h00;
36            end
37        else begin
38            if (we_b)
39                begin
40                    if (add_a != add_b)
41                        begin
42                            mem[add_b] <= data_b;
43                            read_b <= 8'h00;
44                        end
45                    end
46            else
47                begin
48                    read_b <= mem[add_b];
```



```

49     end
50     end
51 end
52
53 endmodule

```

Listing 2: Testbench Code

```

1  `timescale 1ns/1ns
2  module tb_dual;
3
4      // DUT ports
5      reg      clk, rst, we_a, we_b;
6      reg  [7:0] data_a, data_b, add_a, add_b;
7      wire [7:0] read_a, read_b;
8
9      reg [7:0] ref_mem [0:19];
10
11     // Error counters
12     integer error1, error2;
13     integer i, k;
14
15     integer addr;
16     reg [7:0] raddr;
17
18     // DUT instantiation
19     dual dut(
20         .clk(clk), .rst(rst),
21         .we_a(we_a), .we_b(we_b),
22         .data_a(data_a), .data_b(data_b),
23         .add_a(add_a), .add_b(add_b),
24         .read_a(read_a), .read_b(read_b)
25     );
26
27     // Clock: 100 ns period
28     initial clk = 0;
29     always #50 clk = ~clk;
30
31     // Monitor
32     initial
33         $monitor("Time=%0t | err1=%0d err2=%0d total=%0d",
34             $time, error1, error2, error1 + error2);
35
36     // Tasks
37     task do_reset;
38     begin
39         we_a=0; we_b=0; data_a=0; data_b=0;
40         add_a=0; add_b=0; error1=0; error2=0;
41         rst = 1;
42         for (k = 0; k < 20; k = k+1)
43             ref_mem[k] = 8'h00;
44         @(posedge clk); #1;

```

```

45         @(posedge clk); #1;
46         rst = 0;
47     end
48 endtask
49
50 task write_a;
51     begin
52         we_a = 1; we_b = 0;
53         for (i = 0; i < 15; i = i+1) begin
54             @(posedge clk); #1;
55             add_a = i[7:0];
56             data_a = (2*i) & 8'hFF;
57             ref_mem[i] = (2*i) & 8'hFF;
58         end
59         @(posedge clk); #1;
60         we_a = 0;
61     end
62 endtask
63
64 task write_b;
65     begin
66         we_a = 0; we_b = 1;
67         for (i = 0; i < 10; i = i+1) begin
68             @(posedge clk); #1;
69             addr = 10 + i;
70             add_b = addr & 8'hFF;
71             data_b = (3*addr) & 8'hFF;
72             add_a = 8'hFF;
73             ref_mem[addr] = (3*addr) & 8'hFF;
74         end
75         @(posedge clk); #1;
76         we_b = 0;
77     end
78 endtask
79
80 task read_port_a;
81     begin
82         we_a = 0; we_b = 0;
83         for (i = 0; i < 10; i = i+1) begin
84             raddr = $random % 20;
85             if (raddr > 19) raddr = 19;
86             add_a = raddr;
87             @(posedge clk); #1;
88             @(posedge clk); #1;
89             if (read_a !== ref_mem[raddr]) begin
90                 $display("PORT-A MISMATCH addr=%0d got=%0h expected=%0h",
91                     " ",
92                     raddr, read_a, ref_mem[raddr]);
93                 error1 = error1 + 1;
94             end
95         end
96     end
97 endtask

```



```

95     end
96 endtask
97
98 task read_port_b;
99     begin
100         we_a = 0; we_b = 0;
101         for (i = 0; i < 13; i = i+1) begin
102             raddr = $random % 20;
103             if (raddr > 19) raddr = 19;
104             add_b = raddr;
105             @(posedge clk); #1;
106             @(posedge clk); #1;
107             if (read_b != ref_mem[raddr]) begin
108                 $display("PORT-B MISMATCH addr=%0d got=%0h expected=%0h",
109                     raddr, read_b, ref_mem[raddr]);
110                 error2 = error2 + 1;
111             end
112         end
113     end
114 endtask
115
116 // Stimulus
117 initial begin
118     do_reset();
119     write_a();
120     write_b();
121     read_port_a();
122     read_port_b();
123     #100;
124     if ((error1+error2) == 0)
125         $display("ALL TESTS PASSED      Memory working correctly.");
126     else
127         $display("TESTS FAILED      %0d total mismatches.", error1+
128             error2);
129     $stop;
130 end
131 endmodule

```